REVISION CHEAT SHEET:

Relational Database Design

INTERVIEW-READY NOTES • SQL • ER DIAGRAMS • RELATIONSHIPS

Relational database design is the process of deciding:

- What business information must be stored
- Which tables should exist
- Which columns belong in each table
- How tables relate to each other
- Which keys identify rows
- Which foreign keys enforce relationships
- How to avoid duplicate and inconsistent data
- How to represent one-to-one, one-to-many, many-to-many, and hierarchical relationships

A good relational database should represent the **business clearly and accurately**.

The main idea is:

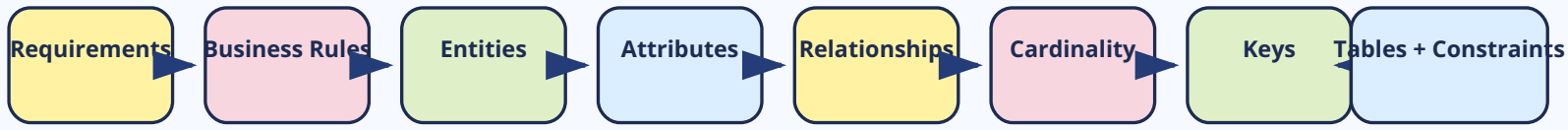
EXAMPLE / FLOW

Business Requirements
↓
Business Rules
↓
Entities
↓
Attributes
↓
Relationships
↓
Cardinality
↓
Keys
↓
Tables + Columns + Constraints

DIAGRAM

Relational Design Journey

Model the business first, then implement the database



From business requirements to SQL Server tables and constraints.

A useful complete example is:

EXAMPLE / FLOW

Customer
↓
Order
↓
OrderItem
↓
Product

This represents a business where customers place orders and each order contains products.

Relational Database

Simple Definition

A **relational database** stores data in tables and connects those tables using relationships.

Example:

EXAMPLE / FLOW

Customers
Orders
Products
OrderItems

Instead of storing everything in one giant table, we divide information into logical tables.

Why Relational Design Matters

Imagine one table like this:

Customer Name	Customer Email	Order ID	Product Name	Product Price
John	john@test.com	101	Laptop	1200
John	john@test.com	101	Mouse	30
John	john@test.com	102	Keyboard	70

Customer information is repeated many times.

Problems include:

EXAMPLE / FLOW

Duplicate data
More storage
Update inconsistencies
Harder maintenance
Incorrect relationships
Poor integrity

A better design separates the information:

EXAMPLE / FLOW

Customers
Orders
Products
OrderItems

Entity

Simple Definition

An **entity** is a real-world thing or business concept about which we want to store data.

Examples:

EXAMPLE / FLOW

Customer
Employee
Product
Order
Department
Payment
Appointment
Course
Student

An entity can represent:

- A person
- A place
- A business object
- An event
- A transaction
- A concept

Entity Example

Suppose we are building an online shopping system.

Important business entities might be:

EXAMPLE / FLOW

Customer
Product
Order
Payment

Conceptually:

EXAMPLE / FLOW

```
Customer
   |
   └── places
         ↓
       Order
```

Entity vs Table

An entity is a **business modeling concept**.

A table is the **physical relational database implementation** of that concept.

Example:

EXAMPLE / FLOW

Business model:
Customer entity
SQL Server implementation:
dbo.Customers table

Think:

EXAMPLE / FLOW

Entity
→ Business concept
Table
→ Database structure storing that entity

They are closely related, but not exactly the same idea.

Attribute

Simple Definition

An **attribute** is a property or characteristic of an entity.

For the Customer entity:

EXAMPLE / FLOW

CustomerId
FirstName
LastName
Email
Phone
DateOfBirth

are attributes.

Diagram:

EXAMPLE / FLOW

```
Customer
|
├── CustomerId
├── FirstName
├── LastName
├── Email
└── Phone
```

Attribute vs Column

An attribute belongs to the conceptual or logical model.

A column is the SQL table implementation.

Example:

EXAMPLE / FLOW

Attribute:
Email
Column:
Customers.Email

Think:

EXAMPLE / FLOW

Entity → Table
Attribute → Column

Relationship

Simple Definition

A **relationship** describes how one entity is connected to another entity.

Examples:

EXAMPLE / FLOW

Customer places Order
Employee belongs to Department
Order contains Product
Student enrolls in Course

Diagram:

EXAMPLE / FLOW

```
Customer
   |
 places
   |
   v
Order
```

Relationship vs Foreign Key

A relationship is a **business concept**.

A foreign key is one common relational database mechanism used to implement and enforce that relationship.

Example business rule:

EXAMPLE / FLOW

Each order belongs to one customer.

Database implementation:

SQL

```sql
CREATE TABLE Orders
(
    OrderId INT PRIMARY KEY,
    CustomerId INT NOT NULL,

    CONSTRAINT FK_Orders_Customers
        FOREIGN KEY (CustomerId)
        REFERENCES Customers(CustomerId)
);
```

Therefore:

EXAMPLE / FLOW

Relationship
→ Business meaning
Foreign Key
→ Database enforcement

Relation

In relational database theory, a **relation** roughly corresponds to a table.

For practical SQL Server learning:

EXAMPLE / FLOW

Relation
≈ Table

A relation contains:

EXAMPLE / FLOW

Rows
+
Columns

Example:

EXAMPLE / FLOW

Customers relation

implemented as:

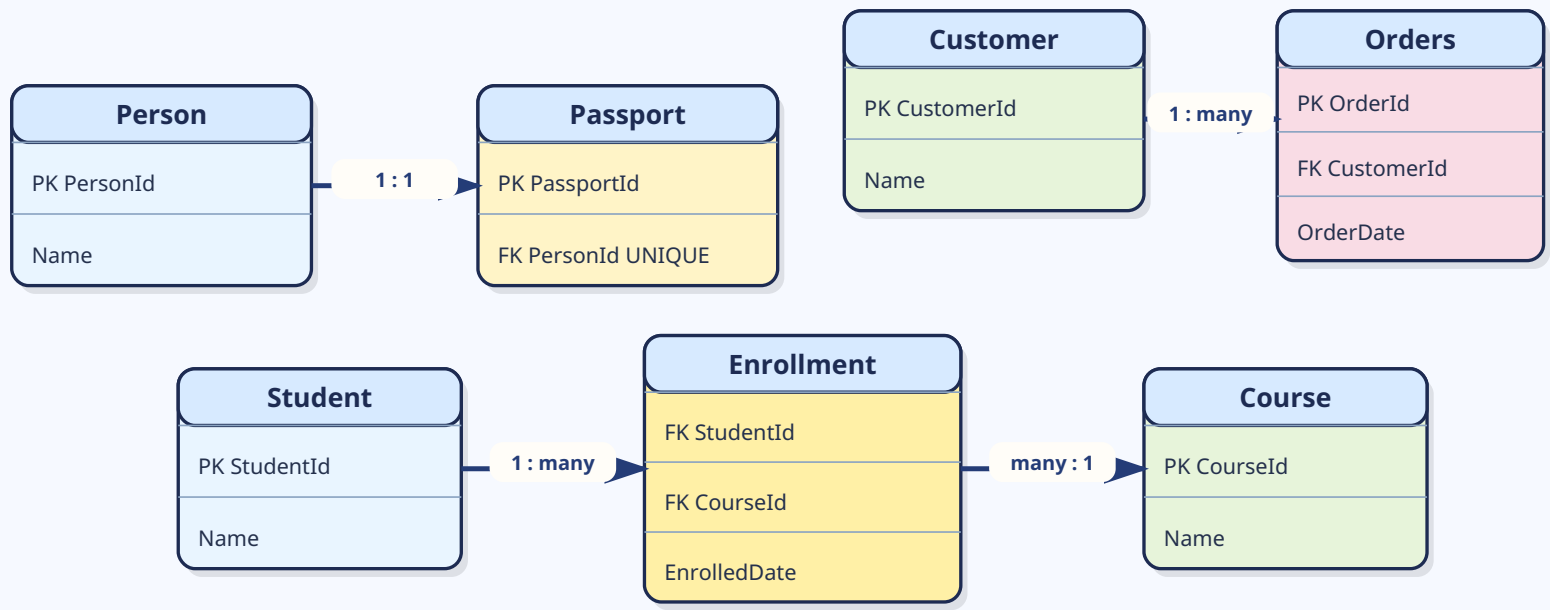
EXAMPLE / FLOW

Customers table

Cardinality

DIAGRAM

Cardinality - 1:1, 1:Many, Many:Many



Many-to-many becomes two one-to-many relationships through a junction table.

Simple Definition

Cardinality describes how many instances of one entity can be related to instances of another entity.

The most important relationship cardinalities are:

EXAMPLE / FLOW

One-to-One
One-to-Many
Many-to-Many

Cardinality answers:

How many?

One-to-One Relationship

Simple Definition

One row in table A is related to at most one corresponding row in table B.

Diagram:

EXAMPLE / FLOW

Person
1
|
|
1
Passport

Example business rule:

EXAMPLE / FLOW

One person has one passport record.
One passport belongs to one person.

One-to-One SQL Example

SQL

```sql
CREATE TABLE Persons
(
    PersonId INT PRIMARY KEY,
    PersonName VARCHAR(100) NOT NULL
);

CREATE TABLE Passports
(
    PassportId INT PRIMARY KEY,

    PersonId INT NOT NULL UNIQUE,

    PassportNumber VARCHAR(30) NOT NULL UNIQUE,

    CONSTRAINT FK_Passports_Persons
        FOREIGN KEY (PersonId)
        REFERENCES Persons(PersonId)
);
```

The important part is:

```sql
PersonId INT NOT NULL UNIQUE
```

The foreign key creates the relationship.

The `UNIQUE` constraint prevents multiple passports from referencing the same person.

When to Use One-to-One

Useful when:

- Data is logically part of the same business object but separated for security
- Optional information should live separately
- A large rarely-used set of columns is separated
- A subtype/extension table is required

Example:

EXAMPLE / FLOW

```
Employee
  1
  |
  1
EmployeeSensitiveData
```

When Not to Use One-to-One

Do not split a table into two one-to-one tables without a reason.

If both sets of columns:

EXAMPLE / FLOW

Always exist together
Have the same security
Are used together
Have the same lifecycle

then one table may be simpler.

One-to-Many Relationship

Simple Definition

One parent row can relate to many child rows.

This is probably the most common relationship in relational databases.

Example:

EXAMPLE / FLOW

Customer
1
|
|
*
Orders

Meaning:

EXAMPLE / FLOW

One customer
→ Many orders
Each order
→ One customer

One-to-Many SQL Example

SQL

```sql
CREATE TABLE Customers
(
    CustomerId INT PRIMARY KEY,
    CustomerName VARCHAR(100) NOT NULL
);

CREATE TABLE Orders
(
    OrderId INT PRIMARY KEY,
    CustomerId INT NOT NULL,

    CONSTRAINT FK_Orders_Customers
        FOREIGN KEY (CustomerId)
        REFERENCES Customers(CustomerId)
);
```

The foreign key is stored on the:

EXAMPLE / FLOW

Many side

So:

EXAMPLE / FLOW

Customers
1
|
*
Orders

means:

EXAMPLE / FLOW

Orders.CustomerId

contains the foreign key.

One-to-Many Rule

A very useful rule:

> In a normal one-to-many relationship, the foreign key is stored in the child table on the many side.

Example:

EXAMPLE / FLOW

Department
1
|
*
Employees

Therefore:

EXAMPLE / FLOW

Employees.DepartmentId

is the foreign key.

Many-to-Many Relationship

Simple Definition

Many rows from entity A can relate to many rows from entity B.

Example:

EXAMPLE / FLOW

Students
*
|
|
*
Courses

Business rule:

EXAMPLE / FLOW

One student can take many courses.
One course can contain many students.

Why Many-to-Many Cannot Usually Be Stored Directly

Suppose:

EXAMPLE / FLOW

Student 1

takes:

EXAMPLE / FLOW

Course 10
Course 20
Course 30

and:

EXAMPLE / FLOW

Course 10

contains many students.

We should not store:

EXAMPLE / FLOW

CourseIds = '10,20,30'

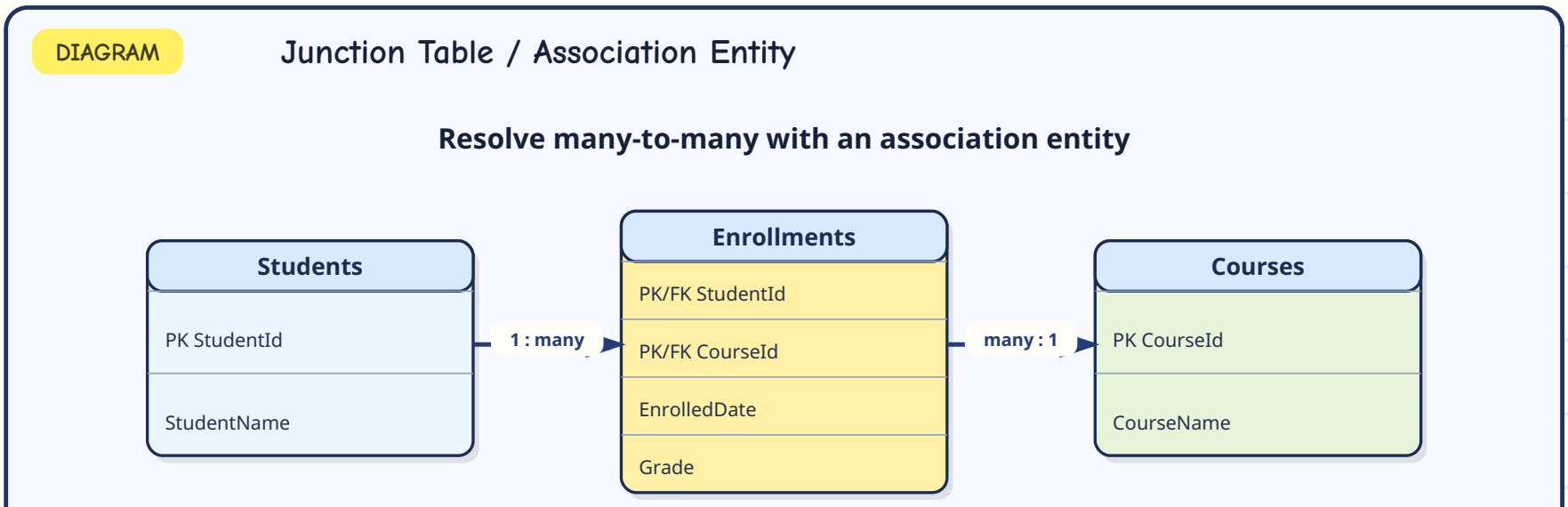
inside the Student table.

That breaks good relational design.

Instead, introduce a third table.

Junction Table

DIAGRAM



Simple Definition

A **junction table** is a table placed between two entities to implement a many-to-many relationship.

Other names include:

EXAMPLE / FLOW

Association table
Bridge table
Link table
Intersection table

Diagram:

EXAMPLE / FLOW

```
Students
   1
   |
   *
Enrollments
   *
   |
   1
Courses
```

The original:

EXAMPLE / FLOW

```
Students * ---- * Courses
```

becomes two one-to-many relationships.

Junction Table Example

SQL

```sql
CREATE TABLE Students
(
    StudentId INT PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL
);

CREATE TABLE Courses
(
    CourseId INT PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL
);

CREATE TABLE Enrollments
(
    StudentId INT NOT NULL,
    CourseId INT NOT NULL,

    EnrolledDate DATE NOT NULL,

    CONSTRAINT PK_Enrollments
        PRIMARY KEY
        (
            StudentId,
            CourseId
        ),

    CONSTRAINT FK_Enrollments_Students
```

```
        FOREIGN KEY (StudentId)
        REFERENCES Students(StudentId),

    CONSTRAINT FK_Enrollments_Courses
        FOREIGN KEY (CourseId)
        REFERENCES Courses(CourseId)
);
```

Why the Junction Table Is Useful

The relationship itself can have attributes.

For example:

EXAMPLE / FLOW

Student + Course

relationship may also have:

EXAMPLE / FLOW

EnrollmentDate
Grade
EnrollmentStatus
CompletionDate

These properties do not belong only to Student.

They also do not belong only to Course.

They belong to:

EXAMPLE / FLOW

Student enrolled in Course

Therefore they belong in:

EXAMPLE / FLOW

Enrollments

Association Entity

When a many-to-many relationship has meaningful attributes, the junction table can also be considered an **association entity**.

Example:

EXAMPLE / FLOW

Student
|
|
Enrollment
|
|
Course

`Enrollment` is more than just two foreign keys.

It represents a real business event.

Bridge Table

A **bridge table** is another common name for a table that connects entities in a many-to-many relationship.

Example:

EXAMPLE / FLOW

Users
|
UserRoles
|
Roles

SQL

```sql
CREATE TABLE UserRoles
(
    UserId INT NOT NULL,
    RoleId INT NOT NULL,

    PRIMARY KEY
    (
        UserId,
        RoleId
    )
);
```

Composite Key in Junction Tables

A **composite key** uses more than one column.

In a junction table:

EXAMPLE / FLOW

StudentId + CourseId

can form the primary key.

Example:

SQL

```
PRIMARY KEY
(
    StudentId,
    CourseId
)
```

This ensures:

EXAMPLE / FLOW

Student 10 + Course 20

cannot appear twice.

Composite Key Junction Design

Example:

EXAMPLE / FLOW

Enrollments

StudentId	CourseId	Grade
1	10	A
1	20	B
2	10	A

Key:

EXAMPLE / FLOW

(StudentId, CourseId)

No separate `EnrollmentId` is necessary if that combination uniquely identifies the relationship and works well for the application.

Surrogate Key in a Junction Table

Sometimes we add:

EXAMPLE / FLOW

EnrollmentId

as a surrogate primary key.

Example:

SQL

```sql
CREATE TABLE Enrollments
(
    EnrollmentId BIGINT
        IDENTITY(1,1)
        PRIMARY KEY,

    StudentId INT NOT NULL,

    CourseId INT NOT NULL,

    EnrolledDate DATE NOT NULL,

    CONSTRAINT UQ_Enrollments_Student_Course
        UNIQUE
        (
            StudentId,
            CourseId
        )
);
```

Now:

EXAMPLE / FLOW

EnrollmentId
→ Surrogate primary key
StudentId + CourseId
→ Business uniqueness

Composite Key vs Surrogate Key in Junction Tables

Composite Primary Key

EXAMPLE / FLOW

PRIMARY KEY(StudentId, CourseId)

Advantages:

- Naturally enforces relationship uniqueness
- No unnecessary ID
- Clear relational meaning

Disadvantages:

- Foreign keys referencing the junction may need multiple columns
- Wider key/index
- Can be inconvenient for some ORMs/APIs

Surrogate Primary Key

EXAMPLE / FLOW

EnrollmentId BIGINT IDENTITY PRIMARY KEY

Advantages:

- Simple single-column identifier
- Easy to reference
- Convenient for APIs and ORMs
- Helpful if the relationship becomes a first-class entity

But you should often still add:

SQL

```sql
UNIQUE(StudentId, CourseId)
```

if duplicate enrollment is not allowed.

Choosing Composite vs Surrogate in a Junction Table

Use a composite key when:

- The relationship itself is simple
- The combination naturally identifies the row
- Other tables rarely need to reference the relationship directly

Consider a surrogate key when:

- The relationship has its own lifecycle
- Other tables reference the relationship
- The junction has many additional attributes
- Application/API simplicity matters

Important:

> Adding a surrogate key does not remove the need for business uniqueness.

Parent and Child Relationships

Parent

The table being referenced.

Child

The table containing the foreign key.

Example:

EXAMPLE / FLOW

```
Customers
   |
   | 1
   |
   | *
Orders
```

Here:

EXAMPLE / FLOW

```
Customers
→ Parent
Orders
→ Child
```

because:

EXAMPLE / FLOW

```
Orders.CustomerId
```

references:

EXAMPLE / FLOW

Customers.CustomerId

Parent/Child SQL Example

SQL

```sql
CREATE TABLE Departments
(
    DepartmentId INT PRIMARY KEY,
    DepartmentName VARCHAR(100)
);

CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,

    DepartmentId INT NOT NULL,

    CONSTRAINT FK_Employees_Departments
        FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
);
```

Referential Integrity

Simple Definition

Referential integrity means relationships between parent and child tables remain valid.

For example:

EXAMPLE / FLOW

Employee.DepartmentId = 50

should only exist if:

EXAMPLE / FLOW

DepartmentId = 50

exists in `Departments`.

The foreign key enforces this.

Invalid Relationship Without a Foreign Key

Imagine:

EXAMPLE / FLOW

```
Departments
DepartmentId
------------
1
2
3
```

but:

EXAMPLE / FLOW

```
Employees
EmployeeId   DepartmentId
------------------------
100          999
```

Department **999** does not exist.

That employee becomes logically inconsistent.

A foreign key prevents this.

Foreign Key Rules

A foreign key typically references:

EXAMPLE / FLOW

Primary Key

or another suitable:

EXAMPLE / FLOW

Unique candidate key

The foreign-key data type should be compatible with the referenced key.

Example:

EXAMPLE / FLOW

Departments.DepartmentId INT
Employees.DepartmentId INT

is good.

Avoid designs such as:

EXAMPLE / FLOW

Departments.DepartmentId INT
Employees.DepartmentId VARCHAR(20)

Optional Relationship

Simple Definition

An optional relationship means the relationship does not have to exist.

Example:

EXAMPLE / FLOW

Employee
may have
Manager

The CEO may have:

EXAMPLE / FLOW

ManagerId = NULL

SQL:

SQL

```sql
ManagerId INT NULL
```

Diagram conceptually:

EXAMPLE / FLOW

```
Employee
  |
  | optional manager
  v
Employee
```

Mandatory Relationship

A mandatory relationship means the child must be associated with the parent.

Example:

EXAMPLE / FLOW

Every Order must belong to a Customer.

Use:

SQL

```sql
CustomerId INT NOT NULL
```

plus the foreign key.

Therefore:

EXAMPLE / FLOW

NOT NULL
→ Relationship is mandatory at column level
NULL allowed
→ Relationship may be optional

Optionality vs Cardinality

These are related but different.

Cardinality asks:

EXAMPLE / FLOW

How many?

Optionality asks:

EXAMPLE / FLOW

Is a relationship required?

For example:

EXAMPLE / FLOW

Customer → Orders

could mean:

EXAMPLE / FLOW

Customer may have zero or many orders.
Order must have exactly one customer.

This can be written conceptually as:

EXAMPLE / FLOW

Customer
1
|
0..*
Orders

Cardinality and Participation

Participation describes whether participation in a relationship is mandatory or optional.

Example:

EXAMPLE / FLOW

Order
must participate in
Customer relationship

because every order requires a customer.

But:

EXAMPLE / FLOW

Customer
does not have to participate in
Order relationship

because a new customer may have no orders yet.

Common Cardinality Notation

Conceptually:

EXAMPLE / FLOW

1
→ exactly one
0..1
→ zero or one
1..*
→ one or many
0..*
→ zero or many

Example:

EXAMPLE / FLOW

Customer 1 ------ 0..* Order

means:

EXAMPLE / FLOW

One customer may have zero or many orders.
Each order belongs to one customer.

Degree of a Relationship

Simple Definition

The **degree** of a relationship describes how many entity types participate in it.

Common degrees:

EXAMPLE / FLOW

Unary
Binary
Ternary

Unary Relationship

One entity type relates to itself.

Example:

EXAMPLE / FLOW

Employee
|
manages
|
Employee

This is also called a:

EXAMPLE / FLOW

Recursive relationship

or:

EXAMPLE / FLOW

Self-referencing relationship

Binary Relationship

Two entity types participate.

Example:

EXAMPLE / FLOW

Customer
|
places
|
Order

This is the most common relationship degree.

Ternary Relationship

Three entity types participate in one business relationship.

Example:

EXAMPLE / FLOW

Doctor
Patient
Medication

Suppose the business concept is:

EXAMPLE / FLOW

Doctor prescribes Medication to Patient

That relationship may involve all three entities.

In physical database design, this may be implemented with an associative table such as:

EXAMPLE / FLOW

Prescription

containing foreign keys to all three entities.

Business Rules

Simple Definition

A **business rule** describes how the business says data should behave.

Examples:

EXAMPLE / FLOW

Every order must belong to one customer.
A customer may place many orders.
Each employee belongs to one department.
One student can enroll in many courses.
The same student cannot enroll in the same course twice.
An employee may have one manager.
A category may have one parent category.

Business rules should drive database design.

Do not start by randomly creating tables.

Start with:

EXAMPLE / FLOW

What does the business require?

Turning Business Rules into Database Design

Business rule:

EXAMPLE / FLOW

One department can contain many employees.

Model:

EXAMPLE / FLOW

Department
1
|
*
Employee

Implementation:

EXAMPLE / FLOW

Employees.DepartmentId
→ Foreign key

Employee-to-Manager Relationship

This is a self-referencing relationship.

Business rule:

EXAMPLE / FLOW

An employee may report to another employee.

Table:

SQL

```sql
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,

    EmployeeName VARCHAR(100)
        NOT NULL,

    ManagerId INT NULL,
```

```
    CONSTRAINT FK_Employees_Manager
        FOREIGN KEY (ManagerId)
        REFERENCES Employees(EmployeeId)
);
```

Employee Hierarchy Diagram

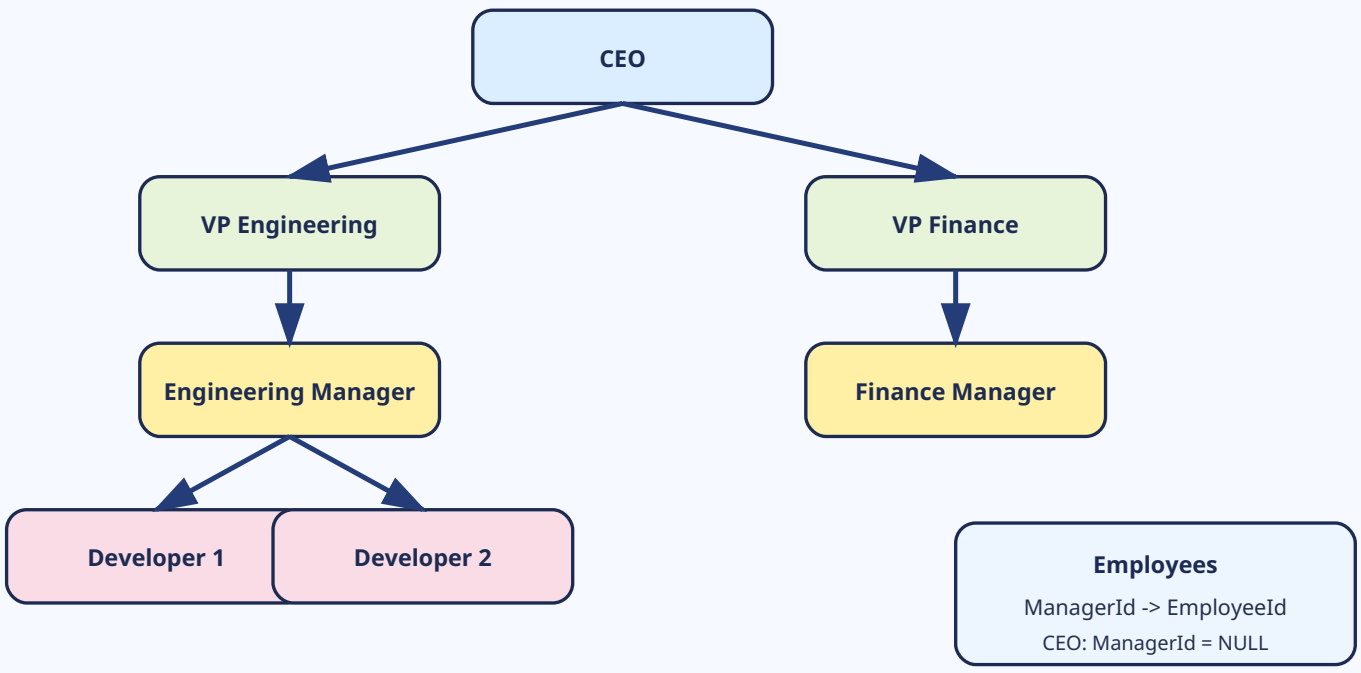
EXAMPLE / FLOW

```
CEO
│
├── VP Engineering
│   ├── Engineering Manager
│   │   ├── Developer 1
│   │   └── Developer 2
│
└── VP Finance
    └── Finance Manager
```

DIAGRAM

Self-Referencing Hierarchy

One table can reference itself through a nullable parent key



Employee-to-manager and category-to-parent designs use the same adjacency-list idea.

In the table:

EXAMPLE / FLOW

```
EmployeeId   EmployeeName          ManagerId
-----------------------------------------------
1            CEO                   NULL
2            VP Engineering        1
3            Engineering Manager   2
4            Developer 1           3
5            Developer 2           3
```

Why ManagerId Can Be NULL

The highest-level employee has no manager.

Therefore:

EXAMPLE / FLOW

```
ManagerId = NULL
```

represents:

EXAMPLE / FLOW

No parent employee

Category-to-Parent Category

Another common self-referencing hierarchy.

Example:

EXAMPLE / FLOW

```
Electronics
|
├── Computers
|   ├── Laptops
|   └── Desktops
|
└── Phones
```

Table:

SQL

```sql
CREATE TABLE Categories
(
    CategoryId INT PRIMARY KEY,

    CategoryName VARCHAR(100)
        NOT NULL,

    ParentCategoryId INT NULL,

    CONSTRAINT FK_Categories_Parent
        FOREIGN KEY (ParentCategoryId)
        REFERENCES Categories(CategoryId)
);
```

Category Data

EXAMPLE / FLOW

CategoryId	CategoryName	ParentCategoryId
1	Electronics	NULL
2	Computers	1
3	Laptops	2
4	Desktops	2
5	Phones	1

This structure is commonly called an:

EXAMPLE / FLOW

Adjacency List

Each row stores the identifier of its parent.

Organizational Hierarchy

Organizational hierarchies can be modeled similarly.

Example:

EXAMPLE / FLOW

```
Company
│
├── Technology
│   ├── Engineering
│   └── Infrastructure
│
└── Finance
    ├── Accounting
    └── Payroll
```

A simple table could contain:

EXAMPLE / FLOW

```
OrgUnitId
OrgUnitName
ParentOrgUnitId
```

with a self-referencing foreign key.

Entity Relationship Diagram — ER Diagram

Simple Definition

An **ER diagram** visually represents:

EXAMPLE / FLOW

```
Entities
Attributes
Relationships
Cardinality
Optionality
```

It helps design the database before writing SQL.

Simple ER Diagram

EXAMPLE / FLOW

```
+------------------+
|    Customer      |
+------------------+
| PK CustomerId    |
| FirstName        |
| LastName         |
| Email            |
+------------------+
```

```
        |
        | 1
        |
        | 0..*
+------------------+
|     Order        |
+------------------+
| PK OrderId       |
| FK CustomerId    |
| OrderDate        |
| TotalAmount      |
+------------------+
```

Meaning:

EXAMPLE / FLOW

One customer
can have zero or many orders.
Each order
belongs to one customer.

ER Diagram Symbols

Different modeling tools use slightly different notation.

A common notation is **Crow's Foot notation**.

Typical ideas:

EXAMPLE / FLOW

| → one
0 → optional / zero
< → many

Conceptually:

EXAMPLE / FLOW

|---| one-to-one
|---< one-to-many
0---< zero-to-many

The exact graphics vary by modeling tool.

The important part is understanding:

EXAMPLE / FLOW

Minimum cardinality
Maximum cardinality

Entity vs Table in ER Modeling

In conceptual modeling:

EXAMPLE / FLOW

Customer

is an entity.

In the physical SQL Server database:

EXAMPLE / FLOW

dbo.Customers

is a table.

An ER diagram may begin conceptually and later become a logical or physical model.

Attribute vs Column in ER Modeling

Conceptual:

EXAMPLE / FLOW

Customer.Email

is an attribute.

Physical:

SQL

```sql
Email VARCHAR(255) NOT NULL
```

is a column.

Relationship vs Foreign Key in ER Modeling

Conceptual:

EXAMPLE / FLOW

Customer places Order

Physical implementation:

EXAMPLE / FLOW

Orders.CustomerId
→ Customers.CustomerId

using a foreign key.



Identifying Relationship

Simple Definition

An **identifying relationship** exists when the parent's key becomes part of the child's primary key.

Example:

EXAMPLE / FLOW

Order
|
OrderItem

Suppose `OrderItems` uses:

EXAMPLE / FLOW

OrderId + LineNumber

as its primary key.

SQL

```sql
CREATE TABLE OrderItems
(
    OrderId BIGINT NOT NULL,
    LineNumber INT NOT NULL,

    ProductId INT NOT NULL,
    Quantity INT NOT NULL,

    CONSTRAINT PK_OrderItems
        PRIMARY KEY
        (
            OrderId,
            LineNumber
        ),
```

```
    CONSTRAINT FK_OrderItems_Orders
        FOREIGN KEY (OrderId)
        REFERENCES Orders(OrderId)
);
```

Here:

EXAMPLE / FLOW

OrderId

is both:

EXAMPLE / FLOW

Foreign Key
+
Part of Primary Key

This is an identifying relationship.

Why It Is Called Identifying

The child cannot be uniquely identified without the parent's identity.

Example:

EXAMPLE / FLOW

LineNumber = 1

is not unique by itself.

But:

EXAMPLE / FLOW

OrderId = 100
LineNumber = 1

uniquely identifies the line.

The parent's key participates in identifying the child.

Non-Identifying Relationship

Simple Definition

A **non-identifying relationship** exists when the foreign key is not part of the child's primary key.

Example:

SQL

```sql
CREATE TABLE Orders
(
    OrderId BIGINT
        IDENTITY
        PRIMARY KEY,

    CustomerId BIGINT NOT NULL,

    CONSTRAINT FK_Orders_Customers
        FOREIGN KEY (CustomerId)
        REFERENCES Customers(CustomerId)
);
```

Here:

EXAMPLE / FLOW

OrderId

identifies the order.

`CustomerId` creates the relationship, but is not part of the primary key.

Therefore this is:

EXAMPLE / FLOW

Non-identifying

Identifying vs Non-Identifying

Identifying	Non-Identifying
Parent key is part of child PK	Parent key is not part of child PK
Child identity depends on parent	Child has independent identity
Common with dependent/detail rows	Very common in application tables

Choosing a Primary Key

A good primary key should usually be:

EXAMPLE / FLOW

Unique
Stable
Not NULL
Minimal
Easy to reference

Avoid keys that:

EXAMPLE / FLOW

Frequently change
Contain unnecessary business information
Are extremely wide
Contain mutable values

Primary Key Example

Good common design:

SQL

```
CustomerId BIGINT
    IDENTITY(1,1)
    PRIMARY KEY
```

Then business uniqueness can still be enforced separately:

SQL

```
Email VARCHAR(255)
    NOT NULL
    UNIQUE
```

Natural Primary Key

A natural key comes from business data.

Example:

EXAMPLE / FLOW

ISBN
EmployeeNumber
CountryCode

It can work well when the value is:

EXAMPLE / FLOW

Stable
Small
Guaranteed unique
Not sensitive

Surrogate Primary Key

A surrogate key is an artificial identifier.

Example:

EXAMPLE / FLOW

CustomerId
OrderId
EmployeeId

Advantages:

EXAMPLE / FLOW

Stable
Small
Simple
Independent of business changes

This is why surrogate keys are very common in application databases.

Choosing Foreign Keys

A foreign key should point to the row that represents the actual parent business concept.

Example:

EXAMPLE / FLOW

Orders.CustomerId
→ Customers.CustomerId

not:

EXAMPLE / FLOW

Orders.CustomerName
→ Customers.CustomerName

Names:

EXAMPLE / FLOW

Can change
May not be unique
May contain spelling differences

Identifiers are more reliable.

Foreign Key Design Rules

Prefer foreign key columns that are:

EXAMPLE / FLOW

Same or compatible data type
Clearly named
Indexed when workload benefits
NOT NULL when relationship is mandatory
NULL when relationship is genuinely optional

Avoiding Redundant Data

Redundant data means storing the same fact unnecessarily in multiple places.

Bad example:

EXAMPLE / FLOW

Orders

OrderId
CustomerId
CustomerName
CustomerEmail

if `CustomerName` and `CustomerEmail` already live in:

EXAMPLE / FLOW

Customers

Why Redundancy Is Dangerous

Suppose:

EXAMPLE / FLOW

CustomerId = 1

changes email.

Now the email might need updates in:

EXAMPLE / FLOW

Customers
Orders
Invoices
Payments

If one place is missed, the database contains conflicting information.

Better Design

Store:

EXAMPLE / FLOW

Customers

CustomerId
CustomerName
CustomerEmail

and:

EXAMPLE / FLOW

Orders

OrderId
CustomerId

Join when needed.

☆

When Duplicate Data Can Be Intentional

Not every repeated-looking value is automatically wrong.

Sometimes history must preserve what was true at a specific time.

Example:

EXAMPLE / FLOW

Customer changes address today.

Old invoices may need to preserve:

EXAMPLE / FLOW

The billing address used when the invoice was issued.

In that case, storing an address snapshot on the invoice may be correct.

This is historical data, not careless redundancy.

Historical Relationships

Simple Definition

A historical relationship preserves how entities were related during a particular period of time.

Example:

EXAMPLE / FLOW

Employee belonged to Department A
from January through June.
Then Department B
from July onward.

If we only store:

EXAMPLE / FLOW

Employees.DepartmentId

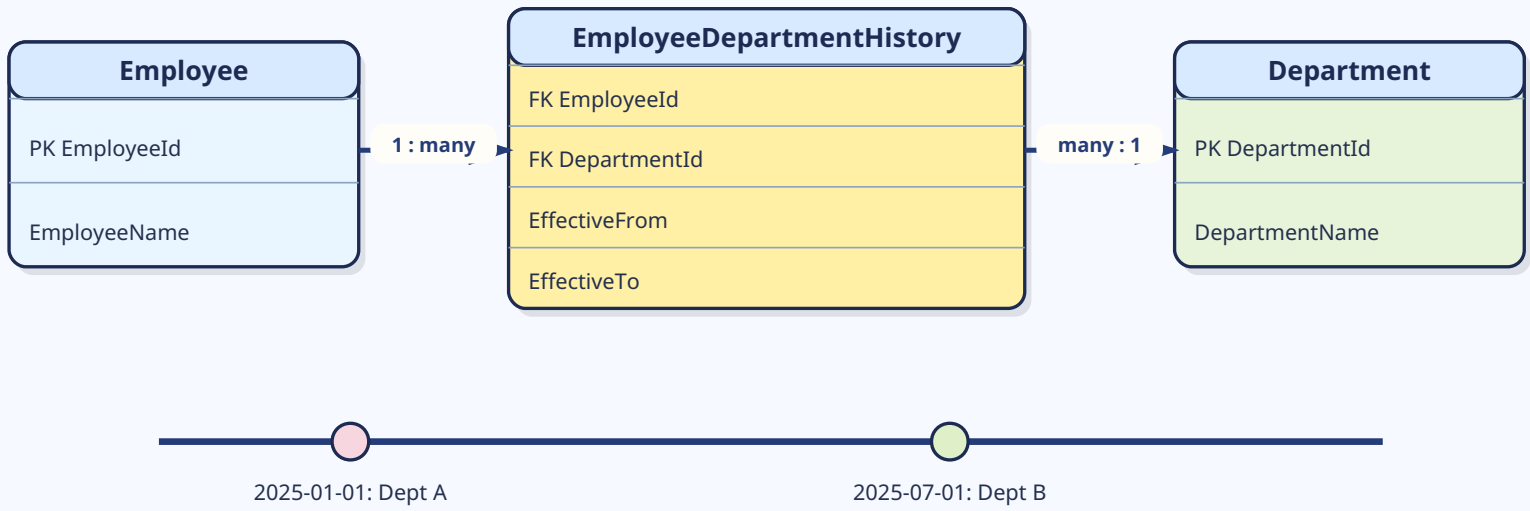
we know only the current department.

We lose history.

Historical Relationship Table

DIAGRAM

Historical Relationship / Effective Dating



Effective dating preserves both past and current relationships

Use EffectiveFrom and EffectiveTo when the business must know how a relationship changed over time.

Better design:

EXAMPLE / FLOW

EmployeeDepartmentHistory

Example:

SQL

```sql
CREATE TABLE EmployeeDepartmentHistory
(
    EmployeeDepartmentHistoryId BIGINT
        IDENTITY
        PRIMARY KEY,

    EmployeeId INT NOT NULL,

    DepartmentId INT NOT NULL,

    EffectiveFrom DATE NOT NULL,

    EffectiveTo DATE NULL,

    CONSTRAINT FK_EDH_Employees
        FOREIGN KEY (EmployeeId)
        REFERENCES Employees(EmployeeId),

    CONSTRAINT FK_EDH_Departments
```

```
        FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId),

    CONSTRAINT CK_EDH_DateRange
        CHECK
        (
            EffectiveTo IS NULL
            OR EffectiveTo >= EffectiveFrom
        )
);
```

Historical Relationship Data

EXAMPLE / FLOW

```
EmployeeId   DepartmentId   EffectiveFrom   EffectiveTo
-----------------------------------------------------
10           1             2025-01-01      2025-06-30
10           2             2025-07-01      NULL
```

This tells us:

EXAMPLE / FLOW

Past

+

Current relationship

When to Use Historical Relationship Tables

Useful for:

EXAMPLE / FLOW

Employee department changes
Employee manager history
Customer account ownership
Contract assignments
Membership history
Pricing relationships
Address history
Role assignments

Effective Dating

Historical tables commonly use:

EXAMPLE / FLOW

EffectiveFrom
EffectiveTo

A NULL `EffectiveTo` can mean:

EXAMPLE / FLOW

Current / still active

Another design may use:

EXAMPLE / FLOW

StartDate
EndDate

The important part is clearly defining the business meaning.

Avoid Overlapping History

Suppose one employee should have only one department at a time.

This would be invalid:

EXAMPLE / FLOW

Department 1
2026-01-01 to 2026-08-31
Department 2
2026-06-01 to 2026-12-31

because the periods overlap.

Preventing overlaps may require more advanced constraints, transactions, procedures, or application logic because simple **CHECK** constraints cannot always compare multiple rows.

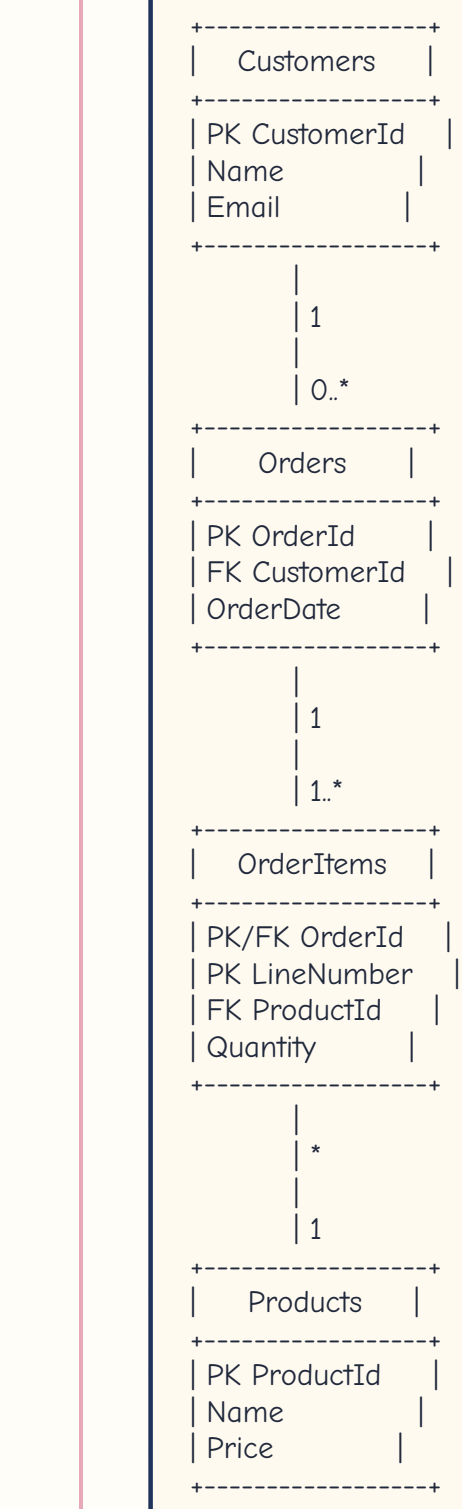
Relationship Types Summary

EXAMPLE / FLOW

One-to-One
One-to-Many
Many-to-Many
Self-Referencing
Identifying
Non-Identifying
Historical

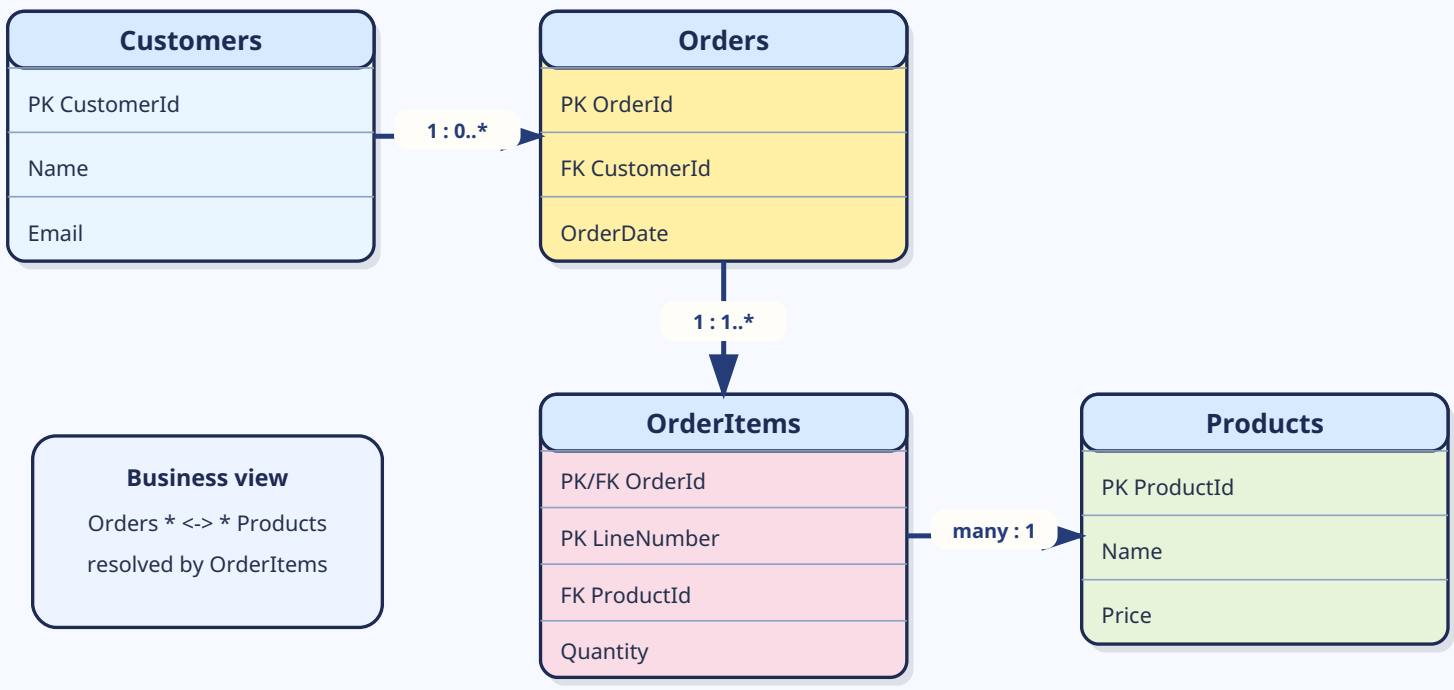
Full E-Commerce ER Example

EXAMPLE / FLOW



DIAGRAM

E-Commerce ER Model



A core relational pattern: parent-child relationships plus a junction/detail table.

This includes:

EXAMPLE / FLOW

Customer → Orders
One-to-many
Order → OrderItems
One-to-many
Product → OrderItems
One-to-many
Orders ↔ Products
Many-to-many implemented through OrderItems

Many-to-Many Hidden Inside the Model

At the business level:

EXAMPLE / FLOW

Orders
*
|
*
Products

because:

EXAMPLE / FLOW

One order has many products.
One product appears in many orders.

Physical implementation:

EXAMPLE / FLOW

Orders
1
|
*
OrderItems
*
|
1
Products

This is one of the most important relational design patterns.

Complete Organization Hierarchy Example

EXAMPLE / FLOW

```
OrganizationUnit
|
├── OrganizationUnitId
├── Name
└── ParentOrganizationUnitId
```

Diagram:

EXAMPLE / FLOW

```
Company
│
├── Technology
│   ├── Engineering
│   │   ├── Backend
│   │   └── Frontend
│   │
│   └── Infrastructure
│
└── Finance
    ├── Accounting
    └── Payroll
```

SQL:

SQL

```sql
CREATE TABLE OrganizationUnits
(
    OrganizationUnitId INT
        IDENTITY
        PRIMARY KEY,

    OrganizationUnitName VARCHAR(100)
        NOT NULL,

    ParentOrganizationUnitId INT NULL,

    CONSTRAINT FK_OrganizationUnits_Parent
        FOREIGN KEY (ParentOrganizationUnitId)
        REFERENCES OrganizationUnits(OrganizationUnitId)
);
```

Database Modeling Process

A practical design process is:

EXAMPLE / FLOW

1. Understand requirements
2. Identify business rules
3. Identify entities
4. Identify attributes
5. Choose candidate keys
6. Choose primary keys
7. Identify relationships
8. Determine cardinality
9. Determine optionality
10. Resolve many-to-many relationships
11. Add foreign keys

12. Add constraints
13. Remove unnecessary redundancy
14. Review historical requirements
15. Create ER diagram
16. Convert model into SQL Server tables
17. Test with realistic data

Design Principle: Model the Business First

Do not start with:

EXAMPLE / FLOW

"I need 20 tables."

Start with:

EXAMPLE / FLOW

What information exists?
What does each thing mean?
How are those things related?
What rules must always be true?

Database structure should come from business meaning.

Design Principle: One Table Should Have a Clear Purpose

Good:

EXAMPLE / FLOW

Customers
→ Customer information
Orders
→ Order information
Payments
→ Payment information

Poor design:

EXAMPLE / FLOW

CustomerOrderProductPaymentEverything

with dozens of unrelated columns.

Design Principle: Store Each Fact in the Correct Place

Ask:

> Which entity does this fact describe?

Example:

EXAMPLE / FLOW

CustomerEmail

belongs to:

EXAMPLE / FLOW

Customer

not:

EXAMPLE / FLOW

Order

But:

EXAMPLE / FLOW

OrderDate

belongs to:

EXAMPLE / FLOW

Order

Design Principle: Avoid Repeating Groups

Bad:

EXAMPLE / FLOW

```
Customer
--------------------------------
CustomerId
Phone1
Phone2
Phone3
Phone4
```

If customers can have many phone numbers, consider:

EXAMPLE / FLOW

```
Customer
   1
   |
   *
CustomerPhone
```

Example:

SQL

```sql
CREATE TABLE CustomerPhones
(
    CustomerPhoneId BIGINT
        IDENTITY
        PRIMARY KEY,

    CustomerId BIGINT NOT NULL,

    PhoneNumber VARCHAR(30) NOT NULL,

    PhoneType VARCHAR(20),

    CONSTRAINT FK_CustomerPhones_Customers
        FOREIGN KEY (CustomerId)
        REFERENCES Customers(CustomerId)
);
```

Design Principle: Use Constraints to Protect Rules

Business rule:

EXAMPLE / FLOW

Email must be unique.

Use:

SQL

```
UNIQUE (Email)
```

Business rule:

EXAMPLE / FLOW

Order must have a customer.

Use:

EXAMPLE / FLOW

CustomerId NOT NULL
+
FOREIGN KEY

Business rule:

EXAMPLE / FLOW

Quantity must be greater than zero.

Use:

SQL

```
CHECK (Quantity > 0)
```

Design Principle: Avoid Meaningless Lookup Explosion

Not every small status or type requires a separate table.

A lookup/reference table is useful when the value has meaningful business metadata or is independently managed.

For a small stable domain:

EXAMPLE / FLOW

Status VARCHAR(20)
CHECK (Status IN ('Active','Inactive'))

may be simpler.

For a richer business concept:

EXAMPLE / FLOW

PaymentMethod
Country
Department
Role

a separate table may make sense.

Design Principle: Think About Delete Behavior

Before defining cascading deletes, ask:

What should happen to children when the parent is deleted?

Possible choices include:

EXAMPLE / FLOW

Reject delete
Cascade delete
Set foreign key to NULL
Archive instead of delete

Business requirements should determine the behavior.

Design Principle: Historical Data Is a Business Decision

Ask:

EXAMPLE / FLOW

Do we only need the current value?
Or must we know what the value was previously?

Examples:

EXAMPLE / FLOW

Current employee department only
vs
Full employee department history

The answer changes the database design significantly.

Common Design Mistakes

Storing Multiple IDs in One Column

Bad:

EXAMPLE / FLOW

CourseIds = '1,2,3,5'

Use a junction table.

Missing Foreign Keys

Without foreign keys, invalid parent-child relationships can enter the database.

Using Names as Foreign Keys

Bad:

EXAMPLE / FLOW

Orders.CustomerName

as the relationship identifier.

Use:

EXAMPLE / FLOW

Orders.CustomerId

Making Everything Nullable

If the business requires a value, use:

EXAMPLE / FLOW

NOT NULL

Optionality should be intentional.

Making Everything NOT NULL

Some relationships are genuinely optional.

Example:

EXAMPLE / FLOW

ManagerId

for the CEO can be **NULL**.

Using a Surrogate Key and Forgetting Business Uniqueness

Example:

EXAMPLE / FLOW

UserId

may be unique while duplicate emails still enter.

Use both:

EXAMPLE / FLOW

PRIMARY KEY(UserId)
UNIQUE(Email)

when required.

Using a Junction Table Without a Uniqueness Rule

Bad:

EXAMPLE / FLOW

StudentId	CourseId
1	10
1	10
1	10

if duplicate enrollment is invalid.

Use:

EXAMPLE / FLOW

PRIMARY KEY(StudentId, CourseId)

or:

EXAMPLE / FLOW

UNIQUE(StudentId, CourseId)

Storing Only Current Relationship When History Is Required

If the business asks:

EXAMPLE / FLOW

Which department was the employee in last year?

a simple:

EXAMPLE / FLOW

Employees.DepartmentId

is insufficient.

Use history/effective-dated modeling.

Master Concept Comparison

Concept	Meaning
Entity	Business thing/concept
Table	Database implementation of entity
Attribute	Property of entity
Column	Database implementation of attribute
Relationship	Business connection
Foreign Key	Enforces relationship
Cardinality	How many
Optionality	Is relationship required
Parent	Referenced table
Child	Table containing FK
Junction Table	Resolves many-to-many
Natural Key	Business-derived identifier
Surrogate Key	Artificial identifier
Composite Key	Multiple columns form key
Referential Integrity	Keeps relationships valid

Concept	Meaning
Identifying Relationship	Parent key is part of child PK
Non-identifying Relationship	Child has independent PK
Historical Relationship	Preserves relationship over time

Easy Memory Rules

EXAMPLE / FLOW

Entity
→ What thing am I storing?

EXAMPLE / FLOW

Attribute
→ What do I know about that thing?

EXAMPLE / FLOW

Relationship
→ How is it connected to another thing?

EXAMPLE / FLOW

Cardinality
→ How many can be connected?

EXAMPLE / FLOW

Foreign Key
→ How do I enforce that connection?

EXAMPLE / FLOW

Junction Table
→ How do I implement many-to-many?

EXAMPLE / FLOW

Optionality
→ Is the relationship required?

EXAMPLE / FLOW

History
→ Do I need only current state or changes over time?

25 Interview Questions

1. What is an entity in database design?

An entity is a real-world object or business concept about which the system stores information, such as Customer, Order, Employee, or Product. In a relational implementation, an entity is commonly represented by a table.

2. What is an attribute?

An attribute describes a property of an entity. For example, `CustomerId`, `FirstName`, and `Email` are attributes of Customer and are usually implemented as columns in the Customers table.

3. What is a relationship?

A relationship describes how entities are associated. For example, Customer places Order. In SQL Server, these relationships are commonly implemented and enforced using foreign keys.

4. What is cardinality?

Cardinality describes the number of entity instances that can participate in a relationship. Common types are one-to-one, one-to-many, and many-to-many.

5. What is a one-to-one relationship?

One row in one entity corresponds to at most one row in another. It is commonly enforced by a foreign key plus a `UNIQUE` constraint on the referencing column.

6. What is a one-to-many relationship?

One parent row can have multiple child rows, while each child references one parent. The foreign key normally appears on the many/child side.

Example:

EXAMPLE / FLOW

Customer 1 → many Orders

7. What is a many-to-many relationship?

Many rows on each side can relate to many rows on the other side. Relational databases normally implement this using a junction table containing foreign keys to both entities.

8. What is a junction table?

A junction table resolves a many-to-many relationship into two one-to-many relationships. It usually contains the foreign keys of the related tables and may contain attributes of the relationship itself.

9. What is the difference between a junction table, association table, and bridge table?

In many practical designs these terms describe the same general pattern: an intermediate table connecting two entities. “Association entity” is especially useful when the relationship itself has meaningful attributes.

10. Should a junction table use a composite key or surrogate key?

Either can be correct. A composite key works well when the foreign-key combination naturally identifies the relationship. A surrogate key is useful when the relationship becomes a first-class entity or other tables need to reference it. If a surrogate key is used, the natural business combination often still needs a `UNIQUE` constraint.

11. What is a parent table and child table?

The parent is the referenced table. The child contains the foreign key referencing the parent.

Example:

EXAMPLE / FLOW

Customers = Parent
Orders = Child

12. What is referential integrity?

Referential integrity guarantees that foreign-key values refer to valid parent rows. It prevents relationships such as an order referencing a customer that does not exist.

13. What is an optional relationship?

An optional relationship means participation is not required. It is often represented using a nullable foreign key, such as an employee whose `ManagerId` may be NULL.

14. What is a mandatory relationship?

A mandatory relationship requires the related parent. It is usually represented by a `NOT NULL` foreign key plus a foreign-key constraint.

Example: every Order must have a Customer.

15. What is the difference between cardinality and optionality?

Cardinality describes how many related rows are allowed. Optionality describes whether at least one relationship is required.

For example:

EXAMPLE / FLOW

Customer → 0..* Orders

means a customer may have zero or many orders.

16. What is a self-referencing relationship?

A self-referencing relationship occurs when a table has a foreign key back to the same table. Examples include Employee-to-Manager and Category-to-ParentCategory relationships.

17. How would you model an employee hierarchy?

Use an Employees table containing a nullable `ManagerId` foreign key that references `Employees.EmployeeId`. Top-level employees such as the CEO have `ManagerId = NULL`.

18. How would you model a category hierarchy?

Use a Categories table with `ParentCategoryId` referencing `CategoryId` in the same table. This adjacency-list design represents parent-child category levels.

19. What is an identifying relationship?

An identifying relationship exists when the parent key becomes part of the child's primary key.

Example:

EXAMPLE / FLOW

OrderId + LineNumber

identifying an OrderItem.

20. What is a non-identifying relationship?

A non-identifying relationship occurs when the child has its own independent primary key and the parent's foreign key is not part of that primary key.

Example:

EXAMPLE / FLOW

Orders.OrderId = PK
Orders.CustomerId = FK

21. How do you choose a good primary key?

A good primary key is unique, stable, not null, and reasonably small. Avoid values that frequently change or contain unnecessary business meaning. Surrogate keys are common, but natural keys can also be appropriate when stable and reliable.

22. Why shouldn't you usually use names as foreign keys?

Names can change, may not be unique, and may contain formatting differences. Stable identifiers such as `CustomerId` or `DepartmentId` are safer relationship keys.

23. Why should redundant data be avoided?

Redundant data increases storage and can create update inconsistencies. A fact should normally have one authoritative location unless duplication is intentionally required for historical or reporting purposes.

24. How do you model a relationship whose value changes over time?

Create a history or effective-dated relationship table containing the related entity keys plus columns such as `EffectiveFrom` and `EffectiveTo`. This preserves both past and current relationships.

25. What is the purpose of an ER diagram?

An ER diagram visually describes entities, attributes, relationships, cardinality, and optionality before or during database implementation. It helps business analysts, developers, and database designers validate that the model correctly represents business rules.

Final Revision Diagram

EXAMPLE / FLOW

Business Rule
↓
Entity
↓
Attributes

↓
Primary Key
↓
Relationships
↓
Cardinality + Optionality
↓
Foreign Keys
↓
Junction Tables where needed
↓
Referential Integrity
↓
ER Diagram
↓
SQL Server Tables

The most important idea from this chapter is:

Relational database design is not mainly about creating tables. It is about accurately modeling business entities, facts, rules, relationships, cardinality, and history, and then implementing that model with appropriate tables, keys, foreign keys, constraints, and junction tables.